CS 422 Assignment 4: Maximum Concurrent Flow and Network Design

CS 422 project team

November 2026

1 Submission and code map

Repository: Assignment 4 folder on GitHub.

- Part 1 topology (generalized de Bruijn): `topologies.py`, lines 10–36.
- Exact concurrent-flow LP: `concurrent_flow.py`, lines 24–69.
- Upper bound $1/h^*(n,d)$: `concurrent_flow.py`, lines 86–95.
- Part 1 evaluation and figures (one-shot: `python run.py`): `run.py`, lines 49–143.
- Part 2 MILP (Gurobi), whole model: `topology_milp.py`, lines 28–69; variables and objective lines 40–43; degree, conservation and capacity constraints lines 45–60.
- Part 2 hose test matrices: `run.py`, lines 45–88; evaluation against fixed topologies: lines 91–113 (one-shot: `python run.py`).

2 Model

There are n nodes and no other switches. Each node has d outgoing and d incoming directed links of capacity 1. Nodes forward traffic for each other, and a demand may be split over several paths. Given a traffic matrix T, the concurrent flow λ is the largest value such that λT_{ij} can be routed from i to j for *every* pair at the same time without exceeding any link capacity.

3 Part 1: uniform traffic

3.1 Upper bound: what any topology can achieve

Under the uniform matrix $T_{ij} = d/(n-1)$, every node sends d in total, so the total demand is nd, which equals the total link capacity nd.

Each hop costs capacity. A unit of flow that travels h hops occupies one unit of capacity on each of h links. Routing λT_{ij} from i to j therefore uses at least $\lambda T_{ij}\,\mathrm{dist}(i,j)$ capacity. Summing over all pairs:

$$\lambda \frac{d}{n-1} \sum_{i \neq j} \mathrm{dist}(i,j) \;\leq\; nd \quad\Longrightarrow\quad \lambda \;\leq\; \frac{n(n-1)}{\sum_{i \neq j} \mathrm{dist}(i,j)} \;=\; \frac{1}{\bar{h}(G)},$$

where $\bar{h}(G)$ is the average shortest-path hop count. For uniform traffic, maximizing λ therefore means minimizing average hop count. The bound is reached exactly only if (a) all flow follows shortest paths and (b) every link is fully loaded, so load must be balanced as well.

No topology has fewer hops than a full d-ary tree. From any node, at most d nodes are 1 hop away, at most d^2 are 2 hops away, and at most d^h are h hops away (the breadth-first argument in the topologies lecture). Filling these layers greedily, $m_h = \min\big(d^h,\ (n-1) - \sum_{k<h} m_k\big)$, gives the smallest possible average hop count $h^*(n,d) = \frac{1}{n-1}\sum_h h\, m_h$, so

$$\lambda \ \le\ 1/\bar{h}(G)\ \le\ 1/h^*(n,d) \qquad \text{for every topology } G.$$

For example, $n = 8, d = 4$ gives layers $(4, 3)$, $h^* = 10/7$, and $\lambda \le 0.7$. Directed graphs that meet this bound exactly do not exist for $d \ge 2$ and diameter ≥ 2 (Plesník and Znám, 1974), so the goal is a construction whose hop counts come close to it.

3.2 Construction: generalized de Bruijn graph

We use the de Bruijn graph from the topologies lecture ("shift left, then append one new digit"). Writing it as arithmetic makes it work for any n:

```
for u in 0 .. n-1:
    for k in 0 .. d-1:
        add link  u -> (d*u + k) mod n
```

Shifting a base-d label left is multiplying by d, and appending digit k is adding k. When $n = d^m$ this is exactly the lecture's de Bruijn graph. Working mod n is needed because the assignment's assumption $(d \mid n-1)$ rules out $n = d^m$. Each node has d outgoing links by construction, and also d incoming links: as u ranges over $0..n-1$ and k over $0..d-1$, the dn values $du+k$ hit every residue mod n exactly d times. Figure 1 shows $n = 9, d = 2$.

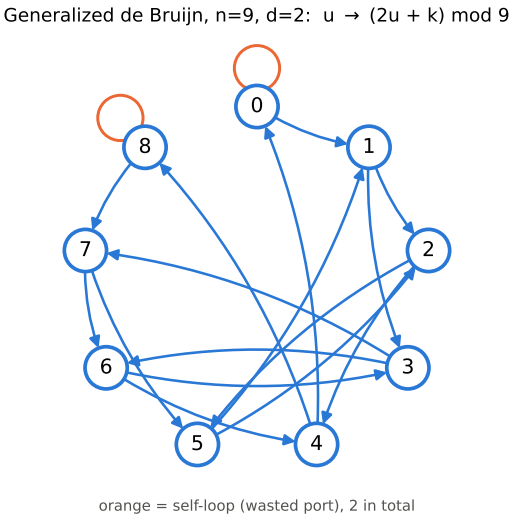


Figure 1: Generalized de Bruijn graph, $n = 9$, $d = 2$. Orange rings are self-loops.

3.3 Why this topology: short paths

Every node reaches every other in at most $\lceil \log_d n \rceil$ hops. After m hops from u with digits $k_1, \ldots, k_m$ we are at

$$d^m u + (k_1 d^{m-1} + k_2 d^{m-2} + \cdots + k_m) \pmod{n}.$$

The bracketed term can be any integer in $0, \ldots, d^m - 1$, the base-d numbers with m digits. Once $d^m \geq n$, these cover every residue mod n, so every target is reachable. This matches the breadth-first lower bound, which says no topology can reach all n nodes in fewer than about $\log_d n$ hops. So de Bruijn's hop counts, and therefore its bound $1/\bar{h}$, are close to the best possible. For comparison, the obvious "ring with chords" (simple circulant $u \to u+1, \ldots, u+d$) reaches only d new nodes per hop, so its average hop count grows linearly in n.

3.4 The gap to the bound

As the lecture notes, "regular labels do not guarantee balanced traffic". De Bruijn misses condition (b) in two ways:

- **Self-loops.** $du + k \equiv u$ can hold. It always holds at $u = 0$ ($k = 0$) and at $u = n - 1$ ($k = d - 1$), and those ports carry nothing.
- **Uneven load.** The graph is not symmetric, so shortest-path routing loads some links more than others. The LP compensates by detouring flow, which costs extra hops.

So we present de Bruijn as *near-optimal with a quantified gap*, not as exactly optimal.

3.5 Evaluation

For each (n, d) with $d \mid n - 1$ we computed the exact λ with a Gurobi LP (one commodity per source, arc flows with conservation and capacity constraints) and compared it with the bound $1/h^*(n, d)$ and with the simple circulant baseline (Figure 2, Table 1).

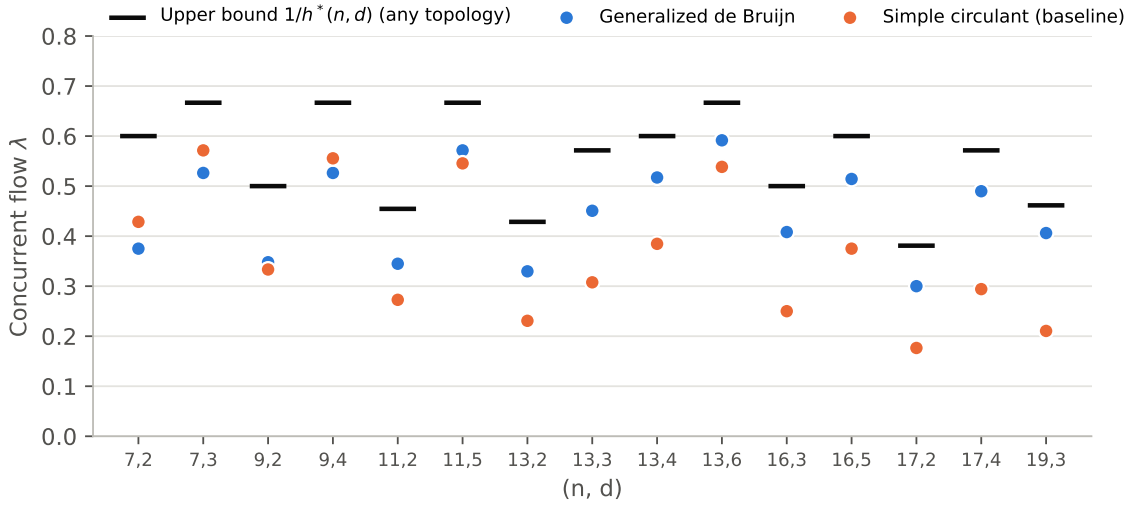


Figure 2: Exact concurrent flow under uniform traffic versus the upper bound that applies to every topology.

Findings.

n	d	bound $1/h^*$	de Bruijn λ	% of bound	max hops	$\lceil \log_d n \rceil$	circulant λ
7	2	0.600	0.375	63	3	3	0.429
7	3	0.667	0.526	79	2	2	0.571
9	2	0.500	0.348	70	4	4	0.333
9	4	0.667	0.526	79	2	2	0.556
11	2	0.455	0.345	76	4	4	0.273
11	5	0.667	0.571	86	2	2	0.545
13	2	0.429	0.330	77	4	4	0.231
13	3	0.571	0.451	79	3	3	0.308
13	4	0.600	0.517	86	2	2	0.385
13	6	0.667	0.592	89	2	2	0.538
16	3	0.500	0.408	82	3	3	0.250
16	5	0.600	0.514	86	2	2	0.375
17	2	0.381	0.300	79	5	5	0.176
17	4	0.571	0.490	86	3	3	0.294
19	3	0.462	0.406	88	3	3	0.211

Table 1: Part 1 results (`part1/results/part1.csv`).

- De Bruijn reaches 63–89% of the bound, which applies to every topology, and 79% or more in 11 of 15 cases (the exceptions are $d = 2$ with $n \leq 13$).
- Its maximum hop count equals $\lceil \log_d n \rceil$ in every case, confirming the argument above.
- It beats the simple circulant in 12 of 15 cases, and the margin grows with n (at $n = 19, d = 3$: 0.406 vs. 0.211). This is the logarithmic vs. linear hop-count difference.
- The circulant wins only for the smallest, densest cases (7/2, 7/3, 9/4). There, both graphs have nearly the same hop counts, and de Bruijn's two or more wasted self-loop ports out of only nd total cost more than its shorter paths save.

4 Part 2: arbitrary hose traffic ($n = 8$, $d = 4$)

We present a **mixed-integer linear program**. Given one traffic matrix $T \in \mathcal{T}_{\text{hose}}$, it chooses the topology and the routing together. The topology *adapts* to T because the program is solved again for each new T, as a reconfigurable (e.g. optical circuit-switched) network would pick a new set of circuits.

4.1 Formulation

Let $V = \{0, \ldots, 7\}$, $A = \{(u, v) : u, v \in V,\ u \neq v\}$ and $d = 4$.

Decision variables.

- $x_{uv} \in \{0, 1, \ldots, d\}$ for $(u, v) \in A$: number of unit-capacity links from u to v, i.e. *the topology*. It is integer because a link cannot be half-built. Values above 1 are parallel links.
- $f^s_{uv} \geq 0$ for $s \in V$, $(u, v) \in A$: traffic that originated at source s and crosses link $u \to v$, i.e. *the routing*.
- $\lambda \geq 0$: the fraction of every demand that is delivered.

Objective. max λ.

Constraints.

$$\sum_{v \neq u} x_{uv} = d \quad \forall u \in V, \qquad \sum_{u \neq v} x_{uv} = d \quad \forall v \in V \quad \text{(degree)} \tag{1}$$

$$\sum_{w \neq v} f^s_{vw} - \sum_{u \neq v} f^s_{uv} = \begin{cases} \lambda \sum_t T_{st} & v = s \\ -\lambda T_{sv} & v \neq s \end{cases} \qquad \forall s, v \in V \qquad \text{(conservation)} \tag{2}$$

$$\sum_{s \in V} f^s_{uv} \leq x_{uv} \qquad \forall (u, v) \in A \qquad \text{(capacity)} \tag{3}$$

4.2 Intuition

- With x *fixed*, constraints (2)–(3) are exactly the concurrent-flow LP we use in Part 1 to evaluate a topology. Source s emits λ times its total demand, each destination v absorbs λT_{sv}, and every other node forwards what it receives (net zero). Flow may be split and may take any number of hops.
- Making x a *variable*, limited only by the port budget (1), lets the solver search over every topology with d outgoing and d incoming links at the same time as the routing. The optimum is therefore the largest λ achievable by *any* such topology for this T. That is exactly "the topology that maximizes concurrent flow for the given traffic matrix".
- The model is linear: λT_{st} is a variable times a constant, and capacity compares flows with x. Only x is integer, so it is a MILP. Gurobi proves optimality (we report its MIP gap).
- Self-loops are excluded because they cannot carry traffic. Parallel links are allowed and necessary: for $T = 4\times$ a permutation, the optimum puts all 4 links on each node's single destination and reaches $\lambda = 1$.
- One commodity per *source* (8 commodities) instead of per pair (56) gives the same λ, because a single-source flow decomposes into paths to each destination. This keeps the model at 56 integer and 449 continuous variables.
- **Upper bound.** For hose matrices with row sums equal to 4, total demand is 32, which equals total capacity $8 \cdot 4$. So $\lambda \leq 1$, with equality only if every demand travels one hop.

4.3 Evaluation

We solved the MILP for 15 hose matrices, each with every row and column summing to 4 and a zero diagonal:

- a $4\times$ permutation;
- two permutations with weight 2;
- three permutations with real weights $(0.5, 0.3, 0.2) \times 4$;
- a skewed matrix (3 to one partner plus 1 spread uniformly);
- the uniform matrix;
- 10 random dense matrices (uniform random entries scaled by Sinkhorn iteration to row/column sums of 4, seed 422).

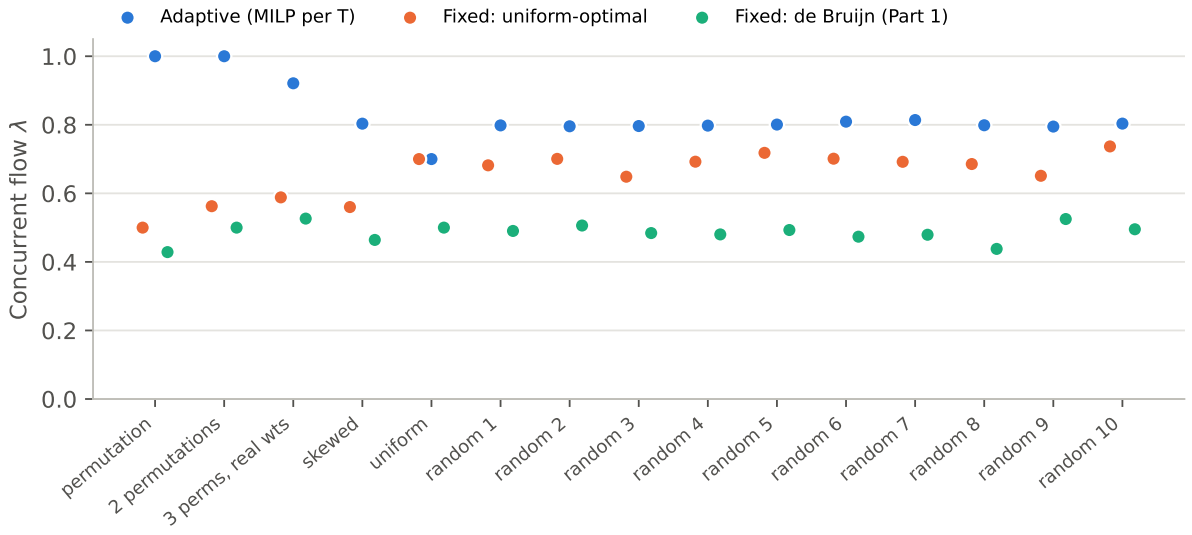


Figure 3: Concurrent flow per hose matrix: topology re-optimized for each T vs. two fixed topologies. All MILP solves reached a gap of at most 4×10^{-5}.

Matrix	adaptive (MILP)	fixed: uniform-optimal	fixed: de Bruijn
4× permutation	1.000	0.500	0.429
2 permutations	1.000	0.563	0.500
3 perms, real weights	0.921	0.588	0.526
skewed	0.803	0.560	0.464
uniform	0.700	0.700	0.500
random (10, mean / range)	0.801 / 0.795–0.814	0.691 / 0.649–0.737	0.487 / 0.438–0.525

Table 2: Part 2 results (`part2/results/part2.csv`).

Every MILP solution was re-scored by the independent Part 1 LP, and the two λ values agree. As baselines that do *not* adapt, we fixed two topologies and evaluated them on every matrix with the same LP: (i) the MILP-optimal topology for the uniform matrix, chosen once, and (ii) Part 1's de Bruijn graph for $n = 8, d = 4$ (Figure 3).

Findings.

- **Integer-structured traffic is served perfectly.** If T is an integer combination of permutations, the MILP builds exactly those direct links (with parallel links where needed) and reaches $\lambda = 1$, the upper bound.
- **Fractional traffic needs some relaying.** For the three-permutation matrix (Figure 4), each demand of 2.0 gets 2 links, but demands of 1.2 and 0.8 can only get 1 link each, since links are whole units and each node has only 4. Part of every 1.2 demand must then travel two hops, using spare capacity on other links (such as the 0.8 links). Two-hop flow costs double capacity, so $\lambda = 0.921 < 1$.
- **Uniform is the hardest case.** The MILP reaches $\lambda = 0.7$, exactly Part 1's bound $1/h^*(8, 4)$, which shows that the bound is tight here. Random dense matrices land in between (≈ 0.80).
- **Adapting pays off.** A fixed topology can be good for one kind of traffic but not for all. The uniform-optimal topology matches the MILP on uniform traffic but delivers only half of a permutation. On random matrices, adapting gives 16% more λ on average. De Bruijn, which is built for uniform

traffic at large n, is worst at $n = 8$. Two of its 32 ports are self-loops and its load is uneven (Part 1, Section 3.4), and at $n = 8$ its short paths give little advantage.

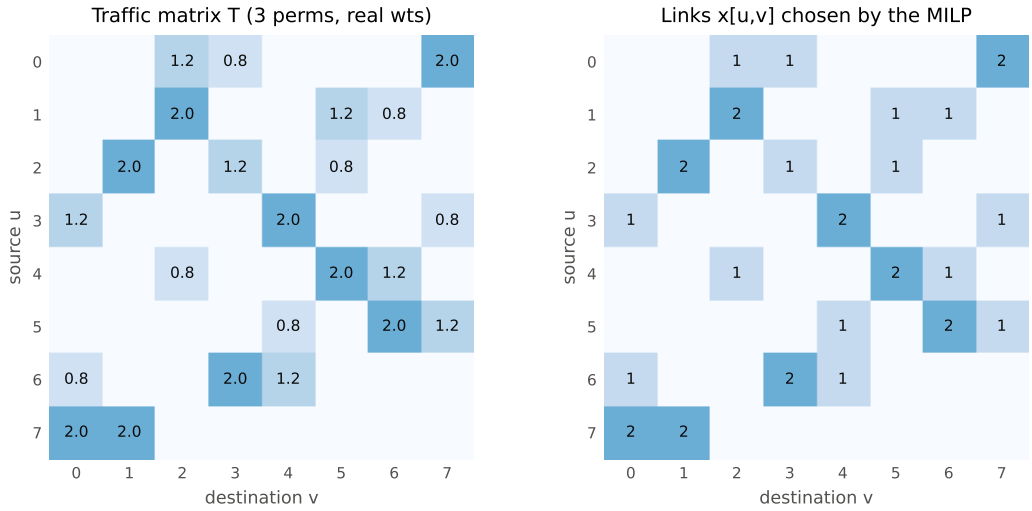


Figure 4: The topology follows the demand: three-permutation matrix (left) and the link counts the MILP chooses for it (right), $\lambda = 0.921$.

5 AI use

We used Claude (Claude Code) interactively, one step at a time, with the group reviewing each step. It helped with: conceptual guidance (concurrent flow, the hop-count bound, mapping lecture slides to the problem), proposing and comparing candidate topologies (the group chose the slide-based de Bruijn construction), drafting the Gurobi LP/MILP code, evaluation scripts and figures, and drafting report text. We ran all code ourselves and validated it against hand-computed cases (complete graph, directed ring, permutation, uniform bound). Decisions and their rationale are logged in `DECISIONS.md`, and AI use in `AI_USAGE.md`.